

Lesson 9 — What Each Notebook Shows

Notebook guide, lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

27 November 2026 · three notebooks

Contents

Notebook 01 — 01_backpropagation_from_scratch	1
Backpropagation from scratch	1
Notebook 02 — 02_keras_softmax_and_depth	2
Keras, softmax, and what depth costs	2
Notebook 03 — 03_training_in_practice	3
Training in practice	3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — 01_backpropagation_from_scratch

Backpropagation from scratch

What it is for. To write a network by hand — forward pass, backward pass, gradient check, training loop — because it is the only way to know what the next two notebooks are calling.

The data. Meridian Instruments tests 3,000 optical sensors on two calibration axes and accepts a unit when **both** offsets are jointly small. 54.7% recorded as accepted, and the test rig itself is wrong **3.3%** of the time, so **0.97 is the ceiling**.

The concepts, in order.

1. **Split first, scale after** — the scaler fitted on the training half alone.
2. **What one straight line can do.** Logistic regression is exactly one neuron: accuracy **0.5507**, against a majority rate of 0.5480. Coefficients almost zero, and it is the right answer to the wrong question — the accept region is a disc. Searched over 72,762 candidate lines, the best line anywhere on this test set reaches only **0.688**, by cutting off a tail rather than enclosing anything.

3. **The forward pass**, with shapes printed, and an untrained cost of 0.6721 against $\log 2 = 0.6931$ for guessing.
4. **The backward pass**, and then **the gradient check**, which is the habit worth teaching: 21 partial derivatives compared against finite differences, **worst relative disagreement 2.0×10^{-5}** , verdict PASS. An analytic gradient that is wrong still trains — badly and silently.
5. **Training, and what four hidden units do**. Five learning rates, and the column to read is *runs ending above their own minimum*: 1/8 at $\eta = 0.05$, **5/8 at $\eta = 0.5$** , 6/8 at $\eta = 1.0$. One run reaches 0.2564 at epoch 268 and ends at **0.8737** — it found a good solution and left it.
6. **The result**: four hidden units, **17 parameters**, training cost $0.6113 \rightarrow 0.2479$, test accuracy **0.9307** where logistic regression managed 0.5507. Each hidden unit *is* a line; the output unit combines them into a region.
7. **The smallest problem that needs a hidden layer**. Common-mode against differential drift — an XOR. Logistic regression: **0.5000**. Two hand-built units move the data so the output's sigmoid can finish the job: **0.9938**. And then the distinction that matters: **representable and findable are different properties** — with 2 units only 4 of 20 random starts beat 0.95; with 3, 15 of 20; with 8, all 20.
8. **How many lines does a disc need?** The best fence of H lines, computed by integral and confirmed by sampling (agreeing to 0.0004): 1 line 0.649, 3 lines 0.863, 8 lines 0.957, the true circle 0.970. **Almost all the gain arrives in the first three lines**.
9. **The one initialisation that cannot work**. From zero: accuracy **0.5480** and **1 distinct hidden unit out of 32** — identical weights receive identical gradients for ever. From random: 0.9173 and 32 of 32.

The two-minute version. *We build a network by hand on a problem where the answer is “inside a region”: logistic regression, which is exactly one neuron, scores 0.55 — the majority rate — and the best straight line anywhere reaches only 0.69. Four hidden units and seventeen parameters reach 0.93 against a rig ceiling of 0.97, because each ReLU unit is a line and the output combines them into a fence. We check the gradient against finite differences before trusting it. Then two things worth keeping: on an XOR-shaped problem two units suffice but plain gradient descent finds them in only four runs out of twenty — representable is not findable — and starting all weights at zero collapses 32 units into one, for ever.*

Notebook 02 — 02_keras_softmax_and_depth

Keras, softmax, and what depth costs

What it is for. Nobody writes the next network by hand. This notebook rebuilds notebook 1's network in six lines, shows the two agree, and then uses the library to reach the lesson's headline number.

The concepts, in order.

1. **The same network, in six lines**. Keras with 4 hidden units: **0.9333** against NumPy's 0.9379 average — and the summary reports **17 trainable parameters**, exactly what was counted by hand.
2. **More than two classes: softmax**, on 8×8 digits. And the honesty to say what a point is worth: **one validation example is 0.278 percentage points**.

3. **Depth on an easy problem**, which is the counterweight to the acceptance data:

architecture	parameters	validation
softmax alone, no hidden layer	650	0.9324
one hidden layer of 32	2,410	0.9602
two hidden layers of 64	8,970	0.9676

Multiclass logistic regression is already **within four points** of a two-layer network here — the opposite of the sensor problem, and the reason “deeper is better” is not a rule.

4. **Three activation functions.** Max derivative: sigmoid **0.2500** exactly, tanh 0.9998. Backpropagation multiplies by that derivative once per layer.
5. **Measuring the shrinkage**, six hidden layers, untrained: the gradient at the last layer against the first is **median 3,547×** for sigmoid (range 2,734–4,607 across seeds), **0.6×** for tanh, **0.5×** for ReLU. And it grows as predicted with depth: 9.5 at 2 layers, 169.6 at 4, 3,553 at 6, 46,250 at 8, against 4^{depth} .
6. **What that costs when you actually train.** Same data, same depth, only the activation differs: **sigmoid 0.1000 — chance, never reaching 0.85**; tanh 0.9750 in 3 epochs; ReLU 0.9611 in 9.
7. **How big should the random weights be?** At layer 8, the fraction of tanh units saturated past 0.99: **0.730** with $N(0, 1)$, **0.000** with $N(0, 0.01)$ — and the second is the worse failure, because the signal is gone rather than stuck. Glorot scaling keeps the layer alive.

The two-minute version. *Six lines of Keras reproduce the hand-written network, same parameter count, same accuracy. Then softmax for ten classes — and note that on the digits, logistic regression with no hidden layer is already within four points of a two-layer network, which is the opposite of the sensor problem. The rest is the lesson’s headline: backpropagation multiplies by the activation’s derivative once per layer, the sigmoid’s peaks at one quarter, and across six layers the gradient arrives about 3,500 times weaker at the first layer than at the last. Train that network and it never leaves chance — 0.10 on ten classes — while the same network with tanh or ReLU passes 0.85 within a handful of epochs.*

Notebook 03 — 03_training_in_practice

Training in practice

What it is for. Everything before this was about what a network *can* represent. This is the gap between that and what you actually get — and it is also the notebook that teaches them to judge whether a difference is a result.

The measuring stick, stated first: on 360 validation examples the binomial standard error near 95% is **1.15 points**, so two methods a point apart are **not distinguishable**.

The concepts, in order.

1. **The learning rate**, three regimes and a narrow useful range: $\eta = 0.001$ gives **0.557** (unfinished, and it looks exactly like a network lacking capacity), $\eta = 0.5$ gives **0.9685**, $\eta = 2.0$ gives **0.101** — chance.
2. **What a large rate does to ReLU units**. Dead units of 64: 3 at $\eta = 0.001$, **33** at $\eta = 1.0$ — and validation accuracy is barely below its best, because the survivors absorb the work. Dead units are a symptom to read, not automatically a fault.
3. **Optimisers, and the promise notebook 1 left open**. Plain gradient descent **0.9389 ± 0.0184** , worst run **0.9027**; momentum 0.9349; **Adam 0.9485 ± 0.0072 , worst 0.9360**. Read the spread and the worst run before the mean: Adam's advantage is mostly reliability.
4. **A network that fits its training set perfectly**. 301,066 parameters on 300 examples — **1,004 parameters per example** — training accuracy **1.0000**, validation 0.9472. Memorisation and failure to generalise are two different things, and only one of them is a problem.
5. **Three interventions, measured honestly**. Early stopping alone 0.9411; + dropout 0.9483; + L2 0.9478; + both **0.9511**. All three land **0.7 to 1.0 points** above the baseline — which is about one standard error, so the honest verdict is *we cannot tell them apart here*.
6. **The intervention that is not a hyperparameter**: 100 examples 0.9028, 300 0.9463, **1,077 0.9787**. Going from 300 to 1,077 examples is worth **+3.2 points**; the best regulariser at 300 was worth **+1.0**.

The two-minute version. *This notebook is about what you actually get, and it opens by computing what a difference has to be before it means anything: on 360 validation points, one percentage point is one standard error. The learning rate takes the same network from unfinished, through working, to chance across three orders of magnitude. At the largest rate half the ReLU layer is dead and the accuracy barely moves, because the survivors take over. Adam's advantage over plain gradient descent is mostly in the worst run, not the mean. Then a network with a thousand parameters per training example: it memorises the training set perfectly, and the three standard regularisers each buy about one point — one standard error, so not distinguishable. Going from 300 training examples to 1,077 buys three points. More data beat every regulariser we tried.*